

T1

[client.py](#)

```
import socket

format = "utf-8"
initial_buffer = 16

port = 5050

hostname = socket.gethostname()
client_ip = socket.gethostbyname(hostname)
server_ip = client_ip

server_sock_addr = (server_ip, port)

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(server_sock_addr)

print("Our client is connected...")

def sending_message(msg):
    message = msg.encode(format)
    message_len = len(message)
    message_len = str(message_len).encode(format)
    message_len += b" "*(initial_buffer - len(message_len))

    client.send(message_len)
    client.send(message)

    sent_from_server = client.recv(2048).decode(format)
    print("Sent from server -->", sent_from_server)

sending_message(f"Device Name: {hostname} & IP_addr: {client_ip}")

while True:
    inp = input("Enter your message: ")
    sending_message(inp)
    if inp == "Terminate":
        break
```

[server.py](#)

```
import socket
import threading

format = "utf-8"
initial_buffer = 16

port = 5050

hostname = socket.gethostname()
server_ip = socket.gethostbyname(hostname)

server_sock_addr = (server_ip, port)

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(server_sock_addr)
server.listen()

print("Our server is listening...")

def handle_clients(conn, client_soc_addr):
    print("#####")
    print("Connected to", client_soc_addr)

    connected = True
    while connected:
        upcoming_message_len = conn.recv(initial_buffer).decode(format)
        print("Sent from client ----> Upcoming message length is",
upcoming_message_len)
        if upcoming_message_len:
            message = conn.recv(int(upcoming_message_len)).decode(format)
            print("Sent from client ----->", message)

            if message == "Terminate":
                conn.send("Bye <3".encode(format))
                connected = False
                print("Connection terminated with", client_soc_addr)
                print("*****")
            else:
```

```
        count = 0
        vowels = "aeiouAEIOU"
        for c in message:
            if c in vowels:
                count+=1

        conn.send(f"Inside {message}; N of vowels are
{count}".encode(format))
        conn.close()

while True:
    conn, client_soc_addr = server.accept()
    thread = threading.Thread(target=handle_clients, args=(conn,
client_soc_addr))
    thread.start()
```

T2

[client.py](#)

```
import socket

format = "utf-8"
initial_buffer = 16

port = 5050

hostname = socket.gethostname()
client_ip = socket.gethostbyname(hostname)
server_ip = client_ip

server_sock_addr = (server_ip, port)

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(server_sock_addr)

print("Our client is connected...")

def sending_message(msg):
    message = msg.encode(format)
    message_len = len(message)
    message_len = str(message_len).encode(format)
    message_len += b" "*(initial_buffer - len(message_len))

    client.send(message_len)
    client.send(message)

    sent_from_server = client.recv(2048).decode(format)
    print("Sent from server -->", sent_from_server)

while True:
    inp = input("Enter your message: ")
    sending_message(inp)
    if inp == "Terminate":
        break
```

[server.py](#)

```
import socket
import threading

format = "utf-8"
initial_buffer = 16

port = 5050

hostname = socket.gethostname()
server_ip = socket.gethostbyname(hostname)

server_sock_addr = (server_ip, port)

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(server_sock_addr)
server.listen()

print("Our server is listening...")

def handle_clients(conn, client_soc_addr):
    print("#####")
    print("Connected to", client_soc_addr)

    connected = True
    while connected:
        upcoming_message_len = conn.recv(initial_buffer).decode(format)
        print("Sent from client ----> Upcoming message length is",
upcoming_message_len)

        if upcoming_message_len:
            message = conn.recv(int(upcoming_message_len)).decode(format)
            print("Sent from client ----->", message)

            if message == "Terminate":
                conn.send("Bye <3".encode(format))
                connected = False
                print("Connection terminated with", client_soc_addr)
                print("*****")
```

```
        else:
            count = 0
            vowels = "aeiouAEIOU"
            for c in message:
                if c in vowels:
                    count += 1

            if count == 0:
                reply = "Not enough vowels"
            elif count <= 2:
                reply = "Enough vowels I guess"
            else:
                reply = "Too many vowels"

            conn.send(reply.encode(format))

    conn.close()

while True:
    conn, client_soc_addr = server.accept()
    thread = threading.Thread(target=handle_clients, args=(conn,
client_soc_addr))
    thread.start()
```


T3

[client.py](#)

```
import socket

format = "utf-8"
initial_buffer = 16

port = 5050

hostname = socket.gethostname()
client_ip = socket.gethostbyname(hostname)
server_ip = client_ip

server_sock_addr = (server_ip, port)

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(server_sock_addr)

def sending_message(msg):
    message = msg.encode(format)
    message_len = len(message)
    message_len = str(message_len).encode(format)
    message_len += b" "*(initial_buffer - len(message_len))

    client.send(message_len)
    client.send(message)

    sent_from_server = client.recv(2048).decode(format)
    print("Sent from server -->", sent_from_server)

while True:
    inp = input("Enter your message: ")
    sending_message(inp)
    if inp == "Terminate":
        break
```

client_copy.py

```
import socket

format = "utf-8"
initial_buffer = 16

port = 5050

hostname = socket.gethostname()
client_ip = socket.gethostbyname(hostname)
server_ip = client_ip

server_sock_addr = (server_ip, port)

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(server_sock_addr)

def sending_message(msg):
    message = msg.encode(format)
    message_len = len(message)
    message_len = str(message_len).encode(format)
    message_len += b" "*(initial_buffer - len(message_len))

    client.send(message_len)
    client.send(message)

    sent_from_server = client.recv(2048).decode(format)
    print("Sent from server -->", sent_from_server)

while True:
    inp = input("Enter your message: ")
    sending_message(inp)
    if inp == "Terminate":
        break
```

[server.py](#)

```
import socket
import threading

format = "utf-8"
initial_buffer = 16

port = 5050

hostname = socket.gethostname()
server_ip = socket.gethostbyname(hostname)

server_sock_addr = (server_ip, port)

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(server_sock_addr)
server.listen()

print("Our server is listening...")

def handle_clients(conn, client_soc_addr):
    print("#####")
    print("Connected to", client_soc_addr)

    connected = True
    while connected:
        upcoming_message_len = conn.recv(initial_buffer).decode(format)
        print("Sent from client ----> Upcoming message length is",
upcoming_message_len)

        if upcoming_message_len:
            message = conn.recv(int(upcoming_message_len)).decode(format)
            print("Sent from client ----->", message)

            if message == "Terminate":
                conn.send("Bye <3".encode(format))
                connected = False
                print("Connection terminated with", client_soc_addr)
                print("*****")
```

```
        else:
            count = 0
            vowels = "aeiouAEIOU"
            for c in message:
                if c in vowels:
                    count += 1

            if count == 0:
                reply = "Not enough vowels"
            elif count <= 2:
                reply = "Enough vowels I guess"
            else:
                reply = "Too many vowels"

            conn.send(reply.encode(format))

    conn.close()

while True:
    conn, client_soc_addr = server.accept()
    thread = threading.Thread(target=handle_clients, args=(conn,
client_soc_addr))
    thread.start()
```

T4

[client.py](#)

```
import socket

format = "utf-8"
initial_buffer = 16

port = 5050

hostname = socket.gethostname()
client_ip = socket.gethostbyname(hostname)
server_ip = client_ip

server_sock_addr = (server_ip, port)

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(server_sock_addr)

def sending_message(msg):
    message = msg.encode(format)
    message_len = len(message)
    message_len = str(message_len).encode(format)
    message_len += b" "*(initial_buffer - len(message_len))

    client.send(message_len)
    client.send(message)

    sent_from_server = client.recv(2048).decode(format)
    print("Sent from server -->", sent_from_server)

while True:
    inp = input("Enter hours worked (or type Terminate): ")
    sending_message(inp)
    if inp == "Terminate":
        break
```

[server.py](#)

```
import socket
import threading

format = "utf-8"
initial_buffer = 16

port = 5050

hostname = socket.gethostname()
server_ip = socket.gethostbyname(hostname)

server_sock_addr = (server_ip, port)

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(server_sock_addr)
server.listen()

print("Our server is listening...")

def handle_clients(conn, client_soc_addr):
    print("#####")
    print("Connected to", client_soc_addr)

    connected = True
    while connected:
        upcoming_message_len = conn.recv(initial_buffer).decode(format)
        print("Sent from client ----> Upcoming message length is",
upcoming_message_len)

        if upcoming_message_len:
            message = conn.recv(int(upcoming_message_len)).decode(format)
            print("Sent from client ----->", message)

            if message == "Terminate":
                conn.send("Bye <3".encode(format))
                connected = False
                print("Connection terminated with", client_soc_addr)
                print("*****")
```



```
        else:
            try:
                hours = int(message)
                if hours <= 40:
                    salary = hours * 200
                else:
                    salary = 8000 + (hours - 40) * 300
                reply = f"Salary: Tk {salary}"
            except ValueError:
                reply = "Invalid input. Please send a valid number."

            conn.send(reply.encode(format))

    conn.close()

while True:
    conn, client_soc_addr = server.accept()
    thread = threading.Thread(target=handle_clients, args=(conn,
client_soc_addr))
    thread.start()
```